
SkillRank: Hybrid Search for Agentic AI Skills

Jiawei Guo
University of Washington
jiawei@uw.edu

Anna Huang
University of Washington
anna1018@uw.edu

Luis Gabriel Cainglet
University of Washington
cainglet@uw.edu

Pushkar Rimmalapudi
University of Washington
rpushkar@uw.edu

Abstract

SkillRank is an intent-based search system for discovering AI agent skills from the `skills.sh` ecosystem. We construct a corpus of roughly 900–1,000 skills containing names, descriptions, documentation text, and metadata such as install counts and recency. Given a natural-language query, the system retrieves candidates using both lexical and semantic retrieval, then reranks them using relevance, title matching, and lightweight popularity signals. To better align casual user phrasing with technical skill descriptions, SkillRank optionally applies lightweight LLM-based query expansion with a fallback to the original query. We evaluate the system on a corrected 20-query ground-truth set using Precision@3 , MRR , and NDCG@3 , and analyze how corpus coverage and description quality affect retrieval performance. The resulting interface allows users to search by intent and quickly access ranked skill recommendations.

1 Problem Statement

Users often cannot find the most relevant AI tools and skills for the tasks they are working on. Traditional search and browsing assume that users already know specific tool names or categories, which is time-consuming and inefficient. As a result, useful skills go undiscovered, and users waste time searching, guessing, or trying the wrong tool. We aim to support skill discovery from user intent rather than from exact names or categories.

2 Motivation

As the AI skills ecosystem grows, manual browsing and keyword guessing scale poorly. Discovery based only on names or categories favors already familiar tools and hides better matches. At the same time, users naturally describe what they want in plain language, such as “summarize long PDFs” or “extract tables from research papers.” If the retrieval system only understands exact keywords, it fails to match how users actually seek help. This motivates an intent-based skill search system that can bridge casual user phrasing and technical skill descriptions.

3 Goals

Our goals are threefold: (1) build a hybrid retrieval system that combines lexical search and dense semantic search so that both exact technical terms and paraphrased intent are captured; (2) design a reranking method that prioritizes relevance first and uses popularity and recency only when the

candidate is already relevant; and (3) deliver an end-to-end system including data pipeline, retrieval, reranking, and a lightweight interface for searching agent skills.

4 Data Preprocessing & Preparation

4.1 Dataset

We build SkillRank on a corpus scraped from the public `skills.sh` directory. Each skill is stored as one JSON object in JSONL format with fields supporting both retrieval and ranking, including skill name, description, example usage or documentation text, weekly installs, total installs, first-seen date, and an external link.

The raw scrape contains approximately 981 skills (`skills_raw.jsonl`). The indexed and evaluated corpus used in our experiments contains 921 skills, representing the subset successfully scraped and indexed at the time of evaluation. This reflects a realistic skill discovery setting in which metadata quality varies and descriptions may be incomplete.

4.2 Preprocessing and Data Preparation

Before indexing, we normalize the scraped records to improve retrieval robustness and make ranking signals consistent.

First, we clean textual fields by removing HTML artifacts and encoded entities and by normalizing whitespace. The `skills.sh` content frequently contains encoded characters and inconsistent formatting, which can degrade both lexical tokenization and semantic similarity if left untreated.

Second, we normalize metadata fields. Install counts are converted from compact strings such as “4.2K” into numeric values, and first-seen timestamps are parsed into a standardized date representation to support recency-based ranking features.

Third, we construct the textual representation used for retrieval by combining a skill’s name, description, and, when available, additional documentation or example usage text. We also normalize common query variants such as “frontend,” “front-end,” and “front end” so they match more consistently.

Finally, we generate dense vector embeddings using the sentence-transformer model `all-MiniLM-L6-v2`. These embeddings are stored locally for efficient similarity search at query time.

4.3 Overall Approach

SkillRank is an intent-based search system that maps natural-language user requests to relevant skills in the indexed corpus. For example, a query such as “summarize long PDFs” or “extract tables from research papers” is translated into a ranked list of relevant skills.

At query time, the system can optionally perform lightweight LLM-based query expansion using a local Ollama model. This step rewrites casual user phrasing into terminology that better matches the skill corpus. If the local model is unavailable, the system falls back to the original query.

The final query is processed by both lexical and dense retrieval, then reranked using relevance, title matching, popularity, and recency signals. Results are exposed through an API and a lightweight interface that returns skill cards with metadata and external links.

5 System Design

5.1 Search

The search subsystem is based on the observation that users express intent in natural language rather than in exact technical vocabulary. Existing skills directories largely rely on keyword search, so a raw query such as “build portfolio website” may have little surface overlap with the skills that best satisfy it. To bridge this gap, our system uses query expansion, hybrid retrieval, and reranking.

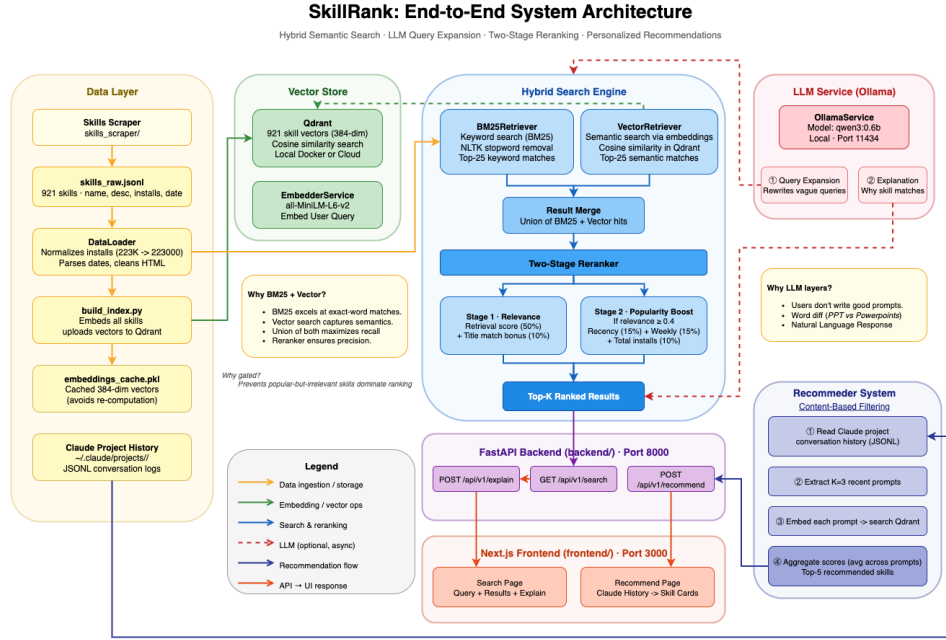


Figure 1: System architecture of SkillRank.

5.1.1 Query expansion

Before retrieval, the user query can be rewritten by a small locally hosted language model through Ollama. Given a user's stated intent, the model produces a concise paraphrase oriented toward skill categories or technical vocabulary. This design is inspired by work on query rewriting for retrieval-augmented systems (Ma et al., 2023). We use a lightweight local model because the task is narrow and benefits from low latency and zero external API cost.

5.1.2 Hybrid retrieval

The expanded query is retrieved against the skill corpus using both BM25 and dense vector retrieval. BM25 captures exact-term matches, which is important when the user specifies framework names or precise terminology (Robertson and Zaragoza, 2009). Dense retrieval complements this by handling paraphrased or concept-level queries through a shared embedding space.

Rather than collapsing the two retrieval signals immediately, the system keeps BM25 and vector scores separately for each candidate. This preserves both lexical and semantic evidence and makes later score combination more stable and interpretable.

5.1.3 Two-stage reranking

Skill popularity varies significantly across the corpus, so naive use of install counts can elevate popular but off-topic skills above niche yet highly relevant ones. To avoid this, the reranker operates in two stages.

Stage 1 scores candidates using retrieval relevance and title matching only. This stage reflects the candidate's pure match to the query.

Stage 2 conditionally adds authority signals for candidates whose Stage 1 relevance exceeds a threshold. These signals include weekly installs, total installs, and recency. Candidates below the threshold are ranked only by Stage 1 relevance. This design lets popularity boost strong matches instead of rescuing weak ones.

6 Evaluation

6.1 Evaluation Setup

We evaluated the search system using a corrected, manually validated test set rather than relying only on the original handwritten expectations. During validation, we found that many originally expected skills were not actually present in the scraped and indexed corpus, which would unfairly penalize the system for failing to retrieve nonexistent items. We therefore rebuilt the ground truth using only real indexed skills and expanded it into a 20-query evaluation set spanning multiple query categories.

Each expected result was assigned a graded relevance label: 2 for exact match, 1 for related, and 0 for not relevant. An evaluation script then issued these queries to the search API and computed ranking metrics automatically.

6.2 Quantitative Results

We report three ranking metrics: Precision@3, Mean Reciprocal Rank (MRR), and NDCG@3. Precision@3 measures how many of the top 3 returned skills are relevant. MRR measures how early the first relevant result appears. NDCG@3 rewards systems that place the most relevant items higher in the ranking.

Across the evaluated queries, the system achieved an average Precision@3 of 0.47, average MRR of 0.63, and average NDCG@3 of 0.50. The strongest results occurred on specific, well-scoped technical queries such as “React Native + Expo” and “Blog post + SEO,” where the system returned highly relevant results near the top.

Query	P@3	MRR	NDCG@3
React Native + Expo	1.00	1.00	0.84
Blog post + SEO	1.00	1.00	0.88
Design system	0.67	1.00	0.87
Figma wireframes	0.67	1.00	0.80
iOS + Firebase	0.33	0.33	0.32
GitHub triage	0.00	0.00	0.00
Average	0.47	0.63	0.50

Table 1: Representative quantitative results and overall averages on the validated evaluation set.

For context, randomly selecting 3 skills from a corpus of this size would produce a Precision@3 near 0.003, so the system performs substantially better than random guessing.

6.3 Qualitative Analysis

Qualitatively, the system performed best when the query was specific and when the relevant skills had strong descriptions in the corpus. For example, React Native and SEO-related queries produced highly relevant top-ranked results.

Most observed failures were driven by data quality rather than by the retrieval pipeline itself. Some failures occurred because relevant skills were not present in the corpus at all, as in the CSV data analysis example. Other failures occurred because the correct skill existed but had weak or missing description text, which reduced both lexical and semantic match quality. In one case (“iOS + Firebase”), the correct skill was retrieved but ranked too low, indicating a ranking issue rather than a retrieval failure.

Overall, these cases suggest that corpus completeness and metadata quality remain major bottlenecks for system performance.

7 Analysis, Trade-offs & Limitations

7.1 Advantages

A major advantage of this system is that it supports intent-based retrieval instead of forcing users to know exact skill names. Compared to keyword-only search systems, it better handles both exact technical terms and paraphrased user intent.

A second advantage is hybrid retrieval. BM25 is effective when users already know precise technical vocabulary, while dense retrieval is more suitable for explanatory or conceptual intents. Combining both improves robustness across different query styles.

A third advantage is the gated use of authority signals. By applying popularity and recency only after a candidate passes a relevance threshold, the system is fairer to niche but highly relevant skills.

7.2 Trade-offs

First, query expansion uses a small local model through Ollama instead of a larger hosted LLM. This reduces latency, removes API cost, and improves privacy because the user’s query does not leave the local environment. However, this choice also limits rewriting quality. A lightweight model may generate less precise reformulations than a stronger hosted model, especially for ambiguous or highly technical requests.

Second, hybrid retrieval improves robustness but adds system complexity, extra storage, and additional latency. A single retrieval method would be simpler to deploy and debug, but would likely perform worse across the wide range of user query styles.

Third, the reranking design intentionally separates relevance from authority. This makes the ranking less biased toward globally popular skills, but it also introduces manual weighting and threshold choices that may not be optimal for every query.

7.3 Limitations

First, the current system depends heavily on the textual quality of the `skills.sh` corpus. If skill descriptions are outdated, inconsistent, or missing, retrieval performance can degrade because both lexical and semantic retrieval rely on informative text. In future work, we plan to mitigate this by enriching the corpus with automated metadata extraction, description normalization, and LLM-based skill summarization.

Second, the current fusion and reranking strategy uses manually chosen weights rather than a learned ranking model. While this design is simple and interpretable, it does not necessarily learn the optimal balance between lexical, semantic, and popularity signals.

Third, the authority weighting scheme is heuristic and may not accurately reflect true skill quality. Signals such as weekly installs, total installs, and recency are only proxy indicators and may not reflect actual usefulness. In future work, we plan to incorporate additional quality signals such as user feedback, success rates, or explicit ratings.

Fourth, the current evaluation remains limited in scale. Although our manually validated 20-query benchmark is substantially more reliable than the original handwritten ground truth, it is still small compared with a large real-world interaction log or user study. In future work, we plan to conduct larger user experiments and collect interaction logs to evaluate the system using real-world usage metrics.

References

- Ma, X., Gong, Y., He, P., Zhao, H., and Duan, N. (2023). Query Rewriting for Retrieval-Augmented Large Language Models. *arXiv preprint arXiv:2305.14283*.
- Robertson, S. E. and Zaragoza, H. (2009). The Probabilistic Relevance Framework: BM25 and Beyond. *Foundations and Trends in Information Retrieval*, 3(4):333–389.